Entregable Lab 04

Ingeniería de Software III - Microservicios con JWT, OpenFeign, Swagger y Docker

Código en GitHub: https://github.com/keevh/sw3-labs/tree/lab-04 (rama lab-04)

Integrantes: 2191918 Kevin Andres Gallardo Robles
2155160 Cristian Eduardo Johnson Acevedo
2162643 Johan Sebastián Villamizar Ortega

Grupo: G8

Fecha: 10 de noviembre de 2026

1. Objetivo y entorno

Para este lab hicimos primero el proyecto base de estudiantes y luego lo partimos en dos microservicios. auth-service hace el login y devuelve un JWT. consulta-service tiene los estudiantes y solo responde si le llega un token que auth-service acepte. Al final corrimos los dos en Docker y probamos desde Swagger.

Herramienta	Versión
Java	21
Spring Boot	3.2.5
Spring Cloud OpenFeign	2023.0.1
JWT	jjwt 0.11.5
Swagger UI (springdoc)	2.6.0
Base de datos	H2 en memoria en cada servicio
Docker / Compose	29.9.1 / 5.6.0

```
$ java -version
openjdk version "21.0.12.1" 2026-08-18
$ mvn -version
Apache Maven 3.10.0 (c43a36b8d67be7e0805a411bc0898af1a51f5472)
$ docker --version
Docker version 29.9.1, build dfc520dcc2
$ docker compose version
Docker Compose version 5.6.0
```

Figura 1. Versiones de Java, Maven y Docker antes de empezar.

Todo quedó en la rama lab-04 con esta estructura. Lo del Lab 03 sigue en su rama.

```
sw3-labs/
  demo-academico/                 proyecto base, puerto 8080
  microservicios-demoacademico/
    auth-service/                 login y JWT, puerto 8081
    consulta-service/             estudiantes, puerto 8082
    docker-compose.yml
```

2. Parte 0: proyecto base

Hicimos demo-academico desde cero con controller, handler, service, repository y model, como dice la guía. Ahora el estudiante tiene apellido y edad, y hay un DTO para crear, otro para actualizar y otro de salida.

El seeder de la guía viene incompleto, el código se corta en el **for**. Lo terminamos nosotros para que cree 500 estudiantes con Faker. Al email le pusimos el número del ciclo, **estudiante0@demoacademico.edu**, **estudiante1@...**, para que no se repitan.

```
$ cd demo-academico && ./mvnw clean package -DskipTests
[INFO] Building demo-academico 0.0.1-SNAPSHOT
[INFO] Building jar: demo-academico/target/demo-academico-0.0.1-SNAPSHOT.jar
[INFO] BUILD SUCCESS
[INFO] Total time:  4.354 s

$ cd microservicios-demoacademico/auth-service && ./mvnw clean package -DskipTests
[INFO] Building auth-service 0.0.1-SNAPSHOT
[INFO] Building jar: microservicios-demoacademico/auth-service/target/auth-service-0.0.1-SNAPSHOT.jar
[INFO] BUILD SUCCESS
[INFO] Total time:  4.062 s

$ cd microservicios-demoacademico/consulta-service && ./mvnw clean package -DskipTests
[INFO] Building consulta-service 0.0.1-SNAPSHOT
[INFO] Building jar: microservicios-demoacademico/consulta-service/target/consulta-service-0.0.1-SNAPSHOT.jar
[INFO] BUILD SUCCESS
[INFO] Total time:  4.197 s

$ ls -lh */target/*.jar */*/target/*.jar
 52M  demo-academico/target/demo-academico-0.0.1-SNAPSHOT.jar
 54M  microservicios-demoacademico/auth-service/target/auth-service-0.0.1-SNAPSHOT.jar
 65M  microservicios-demoacademico/consulta-service/target/consulta-service-0.0.1-SNAPSHOT.jar
```

Figura 2. Compilación de los tres proyectos.

El CRUD lo probé con curl. Crear, editar y borrar funcionaron. Con un email repetido y con el nombre vacío, un email mal escrito y edad 0 me devolvió 400. En el Lab 03 el email repetido daba 409, pero el GlobalExceptionHandler de la guía usa 400 y lo dejamos igual.

```
$ curl "localhost:8080/api/estudiantes?page=0&size=2"
{"success":true,"message":"OK","data":{"content":[{"id":1,"nombre":"Sergio","apellido":"Espinosa",
 "email":"estudiante0@demoacademico.edu","edad":21},{"id":2,"nombre":"Felipe", ... "totalElements":500 ...
HTTP 200

$ curl -X POST localhost:8080/api/estudiantes -d {"nombre":"Kevin","apellido":"Gallardo","email":"kevin@uis.edu.co","edad":22}
{"success":true,"message":"Estudiante creado","data":{"id":501,"nombre":"Kevin","apellido":"Gallardo","email":"kevin@uis.edu.co","edad":22}}
HTTP 201

$ curl -X POST localhost:8080/api/estudiantes -d {... mismo email kevin@uis.edu.co ...}
{"success":false,"message":"Ya existe un estudiante con el email: kevin@uis.edu.co","data":null}
HTTP 400

$ curl -X POST localhost:8080/api/estudiantes -d {"nombre":"","apellido":"X","email":"malo","edad":0}
{"success":false,"message":"Error de validación","data":{"email":"El email no es válido","nombre":"El nombre es obligatorio","edad":"La edad mínima es 1"}}
HTTP 400

$ curl -X PUT localhost:8080/api/estudiantes/501 -d {"nombre":"Kevin Andres","apellido":"Gallardo","email":"kevin@uis.edu.co","edad":23}
{"success":true,"message":"Estudiante actualizado","data":{"id":501,"nombre":"Kevin Andres","apellido":"Gallardo","email":"kevin@uis.edu.co","edad":23}}
HTTP 200

$ curl -X DELETE localhost:8080/api/estudiantes/501
{"success":true,"message":"Estudiante eliminado","data":null}
HTTP 200

$ curl localhost:8080/api/estudiantes/501
{"success":false,"message":"Estudiante no encontrado: 501","data":null}
HTTP 404
```

Figura 3. Pruebas del CRUD en el puerto 8080.

3. Parte 1 y 2: consulta-service y auth-service

consulta-service es el mismo proyecto base copiado, con puerto 8082. En el pom le agregamos OpenFeign y Spring Security, y en el yaml la URL de auth-service en **services.auth.url**.

auth-service va en el 8081 y tiene su propia H2 con la tabla auth_user. Las contraseñas se guardan con BCrypt. Al iniciar crea los tres usuarios de la tabla. En el login se compara la clave con

passwordEncoder.matches y si está bien JwtUtil arma el token con el usuario, el rol y una hora de vigencia.

Usuario	Contraseña	Rol
admin	Admin2026*	ADMIN
docente	Docente2026*	DOCENTE
estudiante1	Estu2026*	ESTUDIANTE

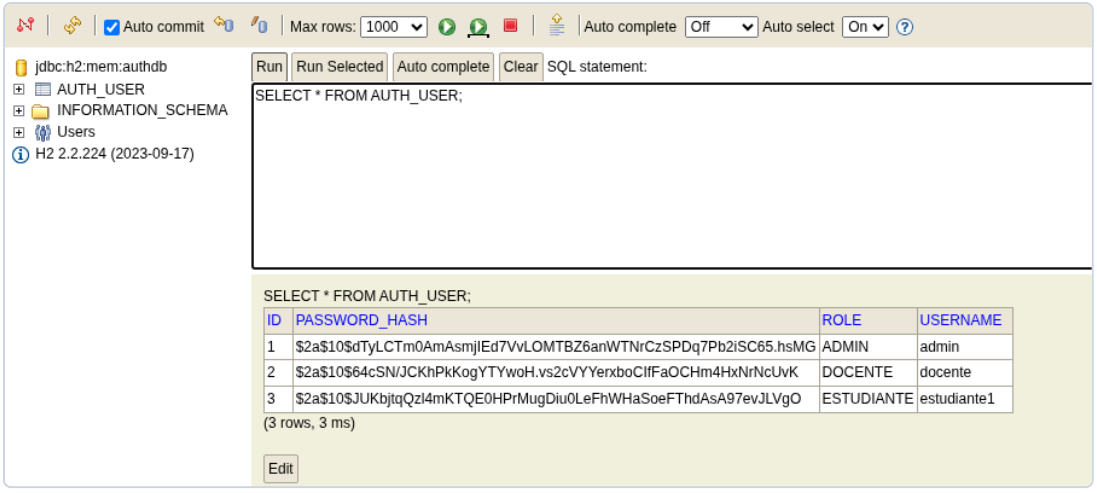


Figura 4. Usuarios en la H2 de auth-service con la contraseña en BCrypt.

4. Parte 3 y 4: OpenFeign y filtro de seguridad

Cuando llega una petición a **/api/estudiantes**, JwtAuthenticationFilter toma el header Authorization y le quita el "Bearer" con TokenUtils. Luego TokenValidationService le pregunta a auth-service por Feign, con AuthFeignClient, si el token sirve. Si no mandaron token o auth-service dice que no es válido, el filtro corta ahí y responde 401.

Con OpenApiConfig aparece el botón Authorize en Swagger. Una cosa que notamos es que el filtro de la guía deja pasar el POST de estudiantes sin token, entonces crear sí se puede sin login. Listar, buscar, editar y borrar sí piden token.

5. Parte 5: Docker

Los Dockerfile copian el JAR, así que antes de **docker compose up --build** hay que hacer **mvnw clean package** en cada servicio. En el compose, consulta-service llama a auth-service con **http://auth-service:8081** porque dentro de Docker localhost es el mismo contenedor. Le quitamos la línea **version: '3.9'** porque Docker Compose ahora muestra un aviso de que está obsoleta.

```
$ docker ps --filter name=service
NAMES              IMAGE                                          STATUS             PORTS
consulta-service   microservicios-demoacademico-consulta-service   Up About a minute   0.0.0.0:8082->8082/tcp, [::]:8082->8082/tcp
auth-service       microservicios-demoacademico-auth-service      Up About a minute   0.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp
```

Figura 5. Contenedores corriendo.

Cuando entramos a la consola H2 con los servicios en Docker salía el mensaje de la figura 6, que las conexiones remotas están deshabilitadas. H2 solo deja entrar desde localhost y desde el navegador uno llega al contenedor por la red de Docker. Lo solucionamos con **spring.h2.console.settings.web-allow-others: true** en los dos servicios. Con eso ya abrió la consola de la figura 4.



Figura 6. Error de la consola H2 antes del cambio.

6. Pruebas en Swagger

Las pruebas las hicimos con los contenedores arriba y en el orden de la guía. Hice login en el Swagger de auth-service, copié el token sin la palabra Bearer y lo pegué en Authorize del Swagger de consulta-service.

Prueba	Esperado	Resultado
1 Login con admin	200 con token	200 token y type Bearer
2 Login con contraseña incorrecta	401	401 Credenciales inválidas
3 Validar el token en /auth/validate	valid true	200 valid true, admin, ADMIN
4 GET /api/estudiantes con token	200	200 con la página de estudiantes
5 GET /api/estudiantes sin autorizar	401	401 Token no enviado o formato inválido
6 GET /api/estudiantes con token inválido	401	401 Token inválido o expirado

Login

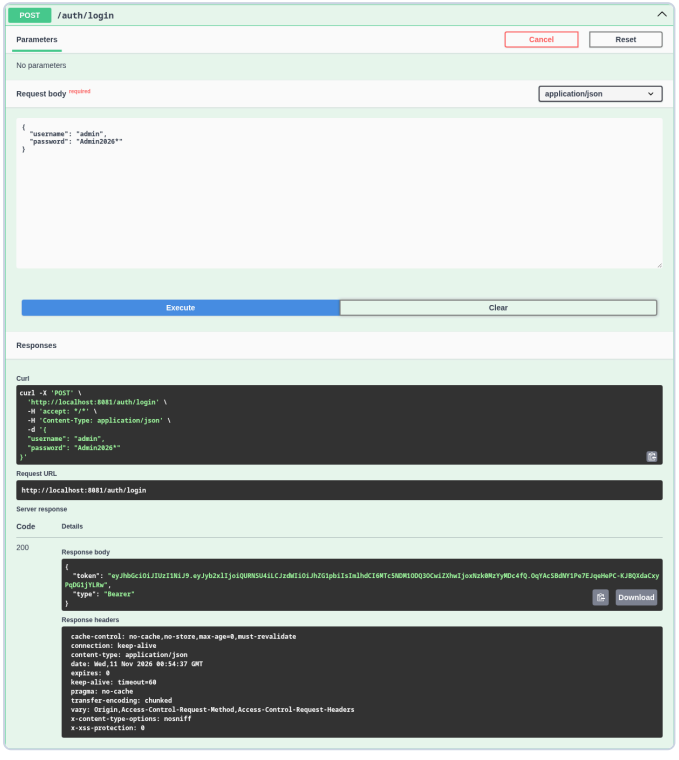


Figura 7. Login con admin, 200 y el token.

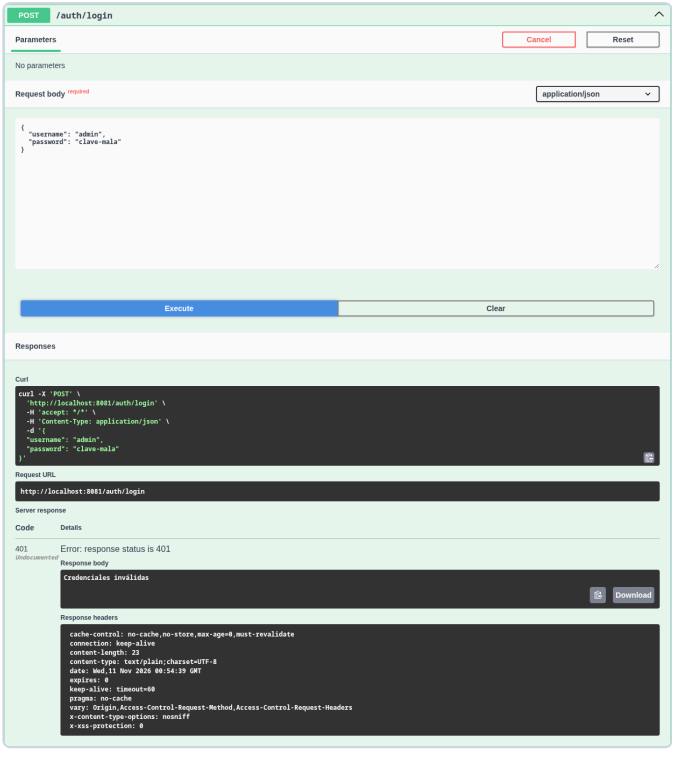


Figura 8. Contraseña mala, 401.

Validar el token

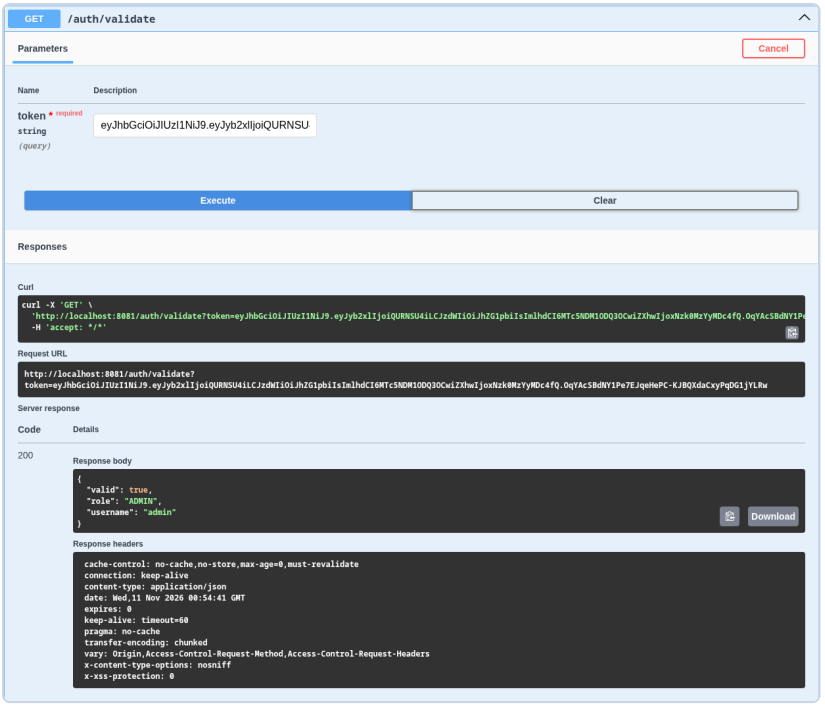


Figura 9. /auth/validate devuelve valid true con el usuario y el rol.

Authorize

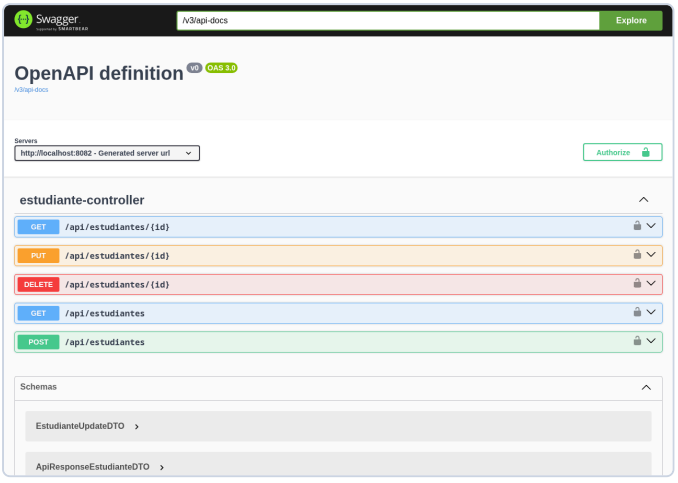


Figura 10. Swagger de consulta-service.

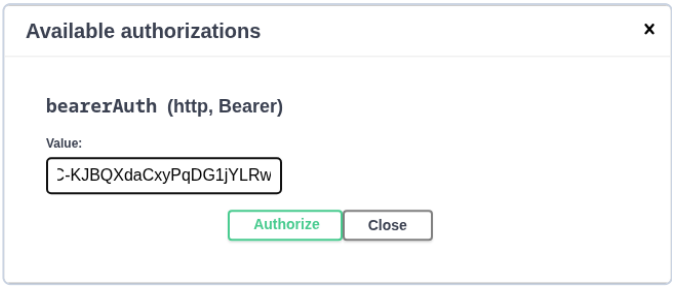


Figura 11. Token pegado en Authorize.

Consultar estudiantes

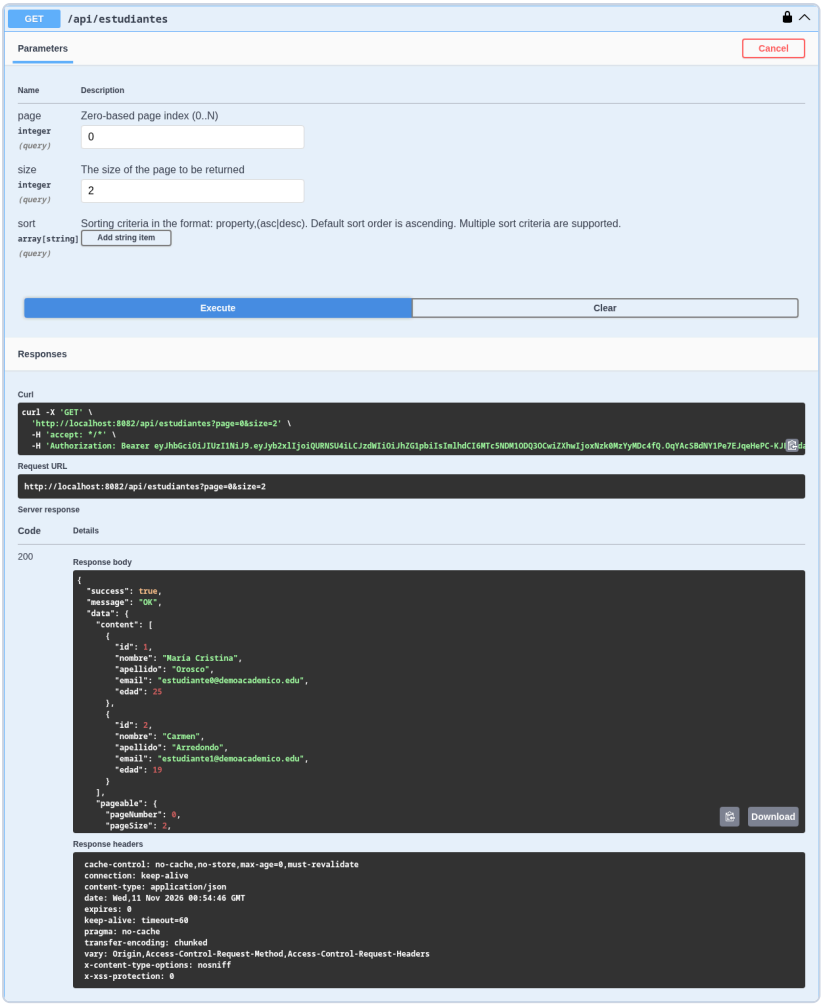


Figura 12. Con token, 200. En el curl se ve que Swagger mandó el header Authorization.

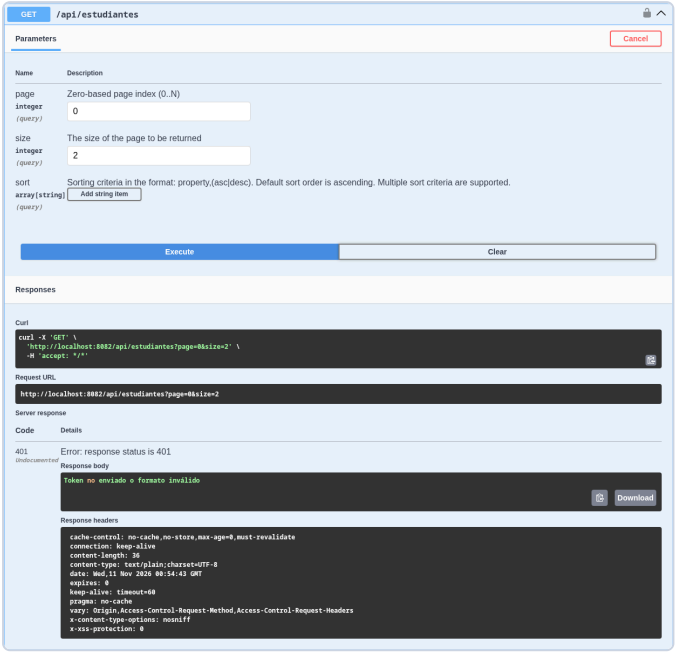


Figura 13. Sin autorizar, 401.

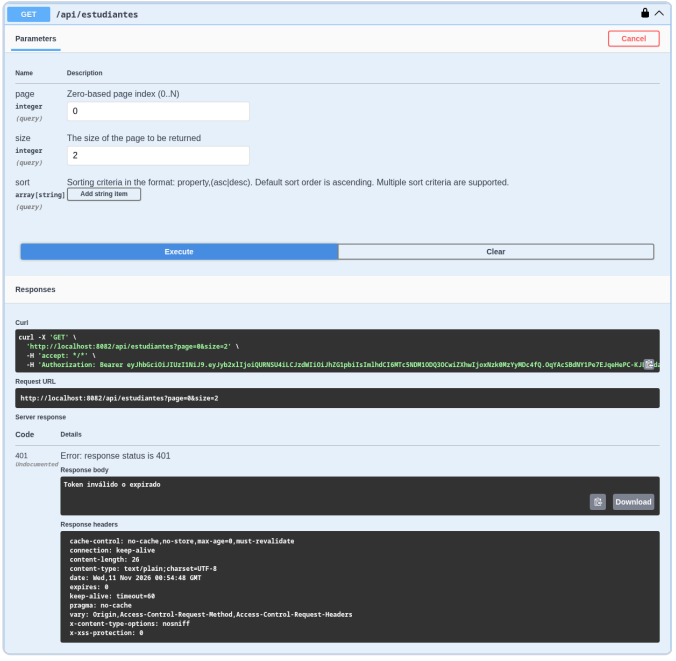


Figura 14. Le cambié los últimos caracteres al token y dio 401.

Las mismas seis pruebas las repetí con curl y dieron los mismos códigos.

```
$ curl -X POST localhost:8081/auth/login -d {"username":"admin","password":"Admin2026*"}
{"token":"eyJhbGciOiJIUzI1NiJ9.eyJyb2xlIjoiQURNSU4...","type":"Bearer"}
HTTP 200

$ curl -X POST localhost:8081/auth/login -d {"username":"admin","password":"clave-mala"}
Credenciales inválidas
HTTP 401

$ curl "localhost:8081/auth/validate?token=<token>"
{"valid":true,"role":"ADMIN","username":"admin"}
HTTP 200

$ curl -H "Authorization: Bearer <token>" "localhost:8082/api/estudiantes?size=2"
{"success":true,"message":"OK","data":{"content":[{"id":1,"nombre":"María Cristina","apellido":"Orosco","email":"estud ...
HTTP 200

$ curl "localhost:8082/api/estudiantes?size=2"   (sin token)
Token no enviado o formato inválido
HTTP 401

$ curl -H "Authorization: Bearer <token alterado>" "localhost:8082/api/estudiantes?size=2"
Token inválido o expirado
HTTP 401
```

Figura 15. Pruebas con curl.

Anexos

Código: https://github.com/keevh/sw3-labs/tree/lab-04, rama lab-04. En el README está cómo correr los servicios local y con Docker Compose, las URLs, los usuarios de prueba y cómo usar Authorize.

Swagger auth-service: http://localhost:8081/swagger-ui.html
Swagger consulta-service: http://localhost:8082/swagger-ui.html
H2 auth-service: http://localhost:8081/h2-console, jdbc:h2:mem:authdb
H2 consulta-service: http://localhost:8082/h2-console, jdbc:h2:mem:demoacademico